

virtspawn

Security & Architecture

Defense-in-depth approach to VM management security

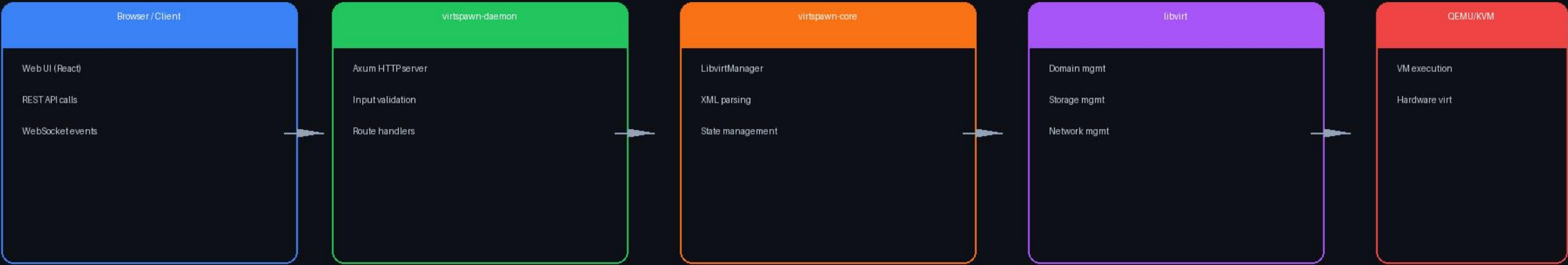
Input validation | SSRF prevention | Audit logging | Panic-free design

Built with Rust for memory safety

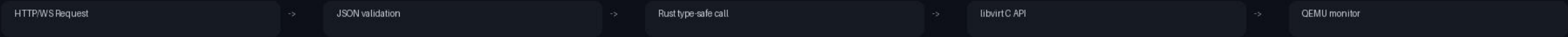
Zero unsafe blocks | No C dependencies in core logic

<https://github.com/ssahani/virtspawn>

System Architecture



Data Flow



Security Boundaries



Deployment: single binary + static web assets | systemd managed | < 10 MB total

Input Validation

VM / Resource Names

`validate_name()`

1-64 characters

Alphanumeric, dash, underscore, dot only

Cannot start with dash or dot

Prevents XML/shell injection in names

vCPU Count

`validate_vcpus()`

Range: 1 - 256

u32 type prevents negative values

Validated before libvirt call

Memory (MB)

`validate_memory_mb()`

Range: 64 MB - 1,048,576 MB (1 TB)

Applied to both `set_memory` and `balloon`

Prevents destabilizing guests with 0 MB

Disk Size (GB)

`validate_disk_gb()`

Range: 1 GB - 10,240 GB (10 TB)

`checked_mul()` prevents integer overflow

`capacity_gb > 0` check in `resize` handler

Migration URI

`validate_migrate_uri()`

Allowed schemes: `qemu://`, `qemu+ssh://`

`qemu+tcp://`, `qemu+tls://`, `qemu+unix://`

Blocks SSRF via `http://`, `file://`, etc.

Network IPs

`validate_ip()` + `validate_subnet()`

DHCP start/end: valid IPv4 format

Subnet 3-octet prefix (e.g. 192.168.100)

Prevents malformed XML injection

Migration SSRF Prevention

Attack Vector

Attacker sends POST /vms/myvm/migrate

`{'dest_uri': 'http://169.254.169.254/metadata'}`

Without validation, daemon connects to

internal AWS/GCP metadata endpoints

or scans internal network ports

Protection

URI scheme allowlist enforced:

`qemu://, qemu+ssh://, qemu+tcp://`

`qemu+tls://, qemu+unix://`

Rejects `http://`, `https://`, `file://`,

`ftp://`, and all non-libvirt schemes

Path Traversal Prevention

CD-ROM ISO Path

Must be absolute path

`canonicalize()` resolves symlinks

Must point to existing regular file

Blocks `/var/./etc/shadow`

Blocks symlinks to sensitive files

Console PTY Path

Must start with `/dev/pts/`

Prevents socat connecting to

arbitrary files or devices

PTY path from libvirt XML only

Existence check before use

XML Escaping

All user inputs XML-escaped before insertion: `<` `>` `'` `"` `&` `<` `>` `'` `"`

Prevents XML injection in VM names, disk paths, network configs, and all libvirt XML operations

 Integer overflow: checked_mul()

 Bounds-checked array access

 let-else instead of unwrap()

 UTF-8 safe string truncation

 No unsafe blocks in core

 Result/Option propagation

Panic Prevention Measures

Storage capacity	capacity_gbchecked_mul(1024*1024*1024) prevents u64 overflow at 16 EB
Template index	templates.get(index) with bounds check, not direct indexing
Form fields	let Some(field) = form.fields.get_mut(i) else { return }
String truncation	s.chars().take(n).collect() instead of &s[..n] byte slicing
Parse errors	Report 'Invalid vCPU count' instead of silently defaulting to 0
Signal handling	Error logging instead of expect() / unwrap() panics
Connection recovery	Close old connection before replacing on reconnect
Pool refresh	Log and continue on non-fatal refresh errors

Rust guarantees: no buffer overflows, no use-after-free, no data races, no null pointer dereferences

Audit Log

Every operation logged with timestamp

Format timestamp action target result

Stored at: ~/virtspawn/audit.log

Logged operations:

create, delete, start, stop, shutdown

reboot, pause, resume, clone, rename

Real-time Monitoring

WebSocket /ws/v1/watch for live events

State changes: running, shutoff, paused

VM added/removed notifications

Heartbeat every 2 seconds

Web UI auto-refreshes on events

HTTP Tracing

tower-http TraceLayer on all requests

Logs: method, path, status, latency

Structured tracing with tracing crate

View with:

journalctl -u virtspawn-daemon -f

RUST_LOG=debug virtspawn-daemon

Metrics Collection

Per-VM: CPU time, memory % disk I/O, net I/O

/api/v1/metrics endpoint (JSON)

Dashboard charts with 30-point history

TUI sparkline trends per VM

No external dependencies (Prometheus-ready)

Why virtspawn?

Feature	virtspawn	virt-manager	Cockpit	Proxmox
Language	Rust + React	Python + GTK	Python + JS	Perl + JS
Web UI	Yes	No (GUI only)	Yes	Yes
Terminal TUI	Yes	No	No	No
REST API	30+ endpoints	No	Limited	Yes
Single binary	Yes (< 10 MB)	No (many deps)	No (many deps)	No (OS distro)
Memory safety	Rust (compile)	Python (runtime)	Python (runtime)	Perl (runtime)
Real-time events	WebSocket	No	Limited	No
VNC in browser	Yes (noVNC)	No	Yes	Yes
Serial console	Yes (WebSocket)	No	Yes	Yes
Input validation	Comprehensive	Basic	Basic	Moderate
Audit logging	Built-in	No	Limited	Yes
Setup time	5 minutes	Package install	Package install	Full OS install
Footprint	< 10 MB	~200 MB	~500 MB	Full OS

virtspawn: Purpose-built, lightweight, secure VM management

No bloat | No full OS required | Production-ready in minutes

Security Summary

1 Rust memory safety - no buffer overflows, use-after-free

2 Input validation on all API endpoints with descriptions

3 Migration URI allowlist prevents SSRF attacks

4 Path canonicalization prevents directory traversals

5 PTY path validation prevents command injection vulnerabilities

6 XML escaping on all user-supplied values

7 No CORS headers - same-origin policy enforced

8 Integer overflow protection with checked arithmetic

9 Panic-free design - no unwrap() on user input

10 Audit logging for all operations with timestamps

11 Graceful error handling with structured error types

12 Connection auto-recovery with explicit cleanup